AgentCgroup

AI Agent Resource Characterization and OS-Level Control

Yusheng Zheng (yzhen165@ucsc.edu)
PhD Student

Roadmap

- Motivation & Problem
- Resource Characterization (144 tasks, 2 models)
- Agent vs. Cloud Workloads & Three Mismatches
- AgentCgroup: eBPF + Bash Wrapper + Negotiation
- Evaluation & Takeaways

Motivation

AI coding agents — **Claude Code, Codex, Devin** — are being deployed at scale, autonomously running compilers, test suites, and package managers in sandboxed containers.

A New Workload Class

- LLM reasons → calls tools → observes → iterates
- Agent **autonomously decides** what to run and when
- Resource demands are **tool-call driven** and unpredictable

An Unstudied Problem

- Cloud providers host **50–100+ agents per machine**
- OS-level resource behavior **completely unknown**
- → **OOM kills, resource waste, poor density**

→ Can we exploit resource patterns for **higher deployment density**?

Characterization: Setup

144 SE tasks from SWE-rebench benchmark, with Claude Code

Config	Model	Tasks
Local GPU	GLM-4.7-Flash	111
Cloud API	Claude Haiku 4.5	33

Hardware: Intel Core Ultra 9 285K, 24 cores, 128 GB DDR5

Environment: Podman containers, Linux 6.15, cgroup v2, 1s sampling

What We Measure

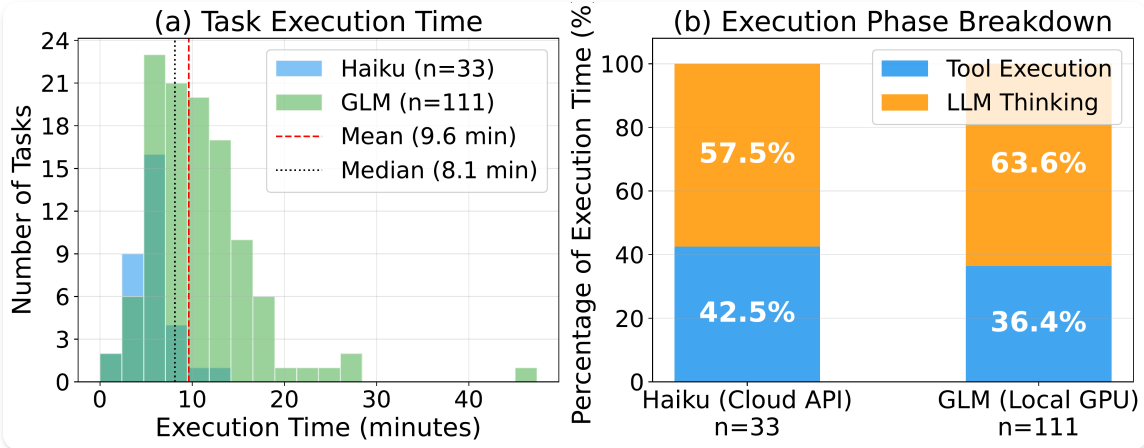
- **Where does time go?** (execution model)
- **What does resource usage look like?** (resource dynamics)
- **Can demands be predicted?** (cross-task variance)

Container Images

- **Agent container images average 3.5 GB, 7× larger than typical microservice images and 70× larger than serverless functions**

Execution Model

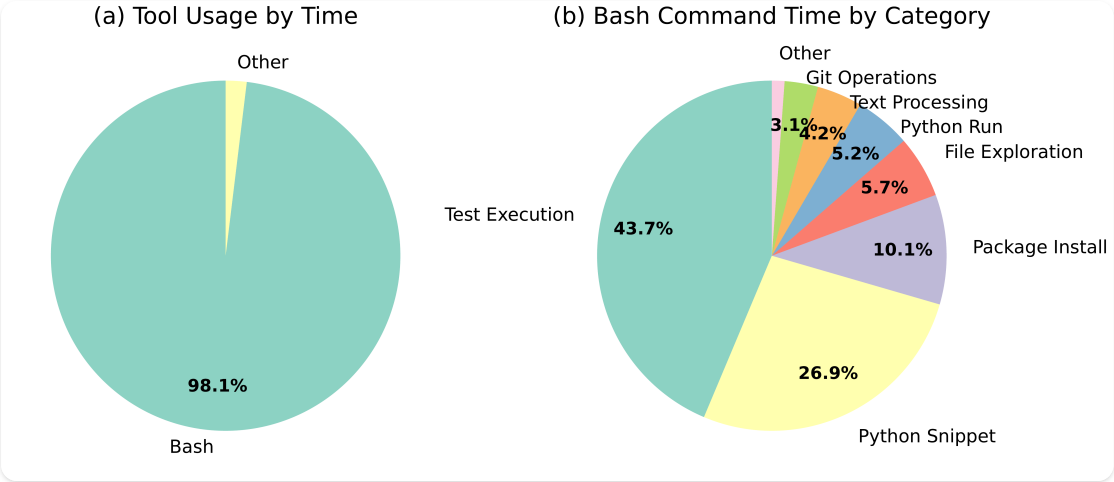
Q: Where does the time go?



- Task duration: median **8.1 min**, mean 9.6 min (Haiku 5.8 min, GLM 10.8 min)
- **LLM reasoning accounts for 26–44% of end-to-end task latency; the remainder is consumed by tool execution (~40% of active time) and initialization (29–45%)**

What Tool Dominates?

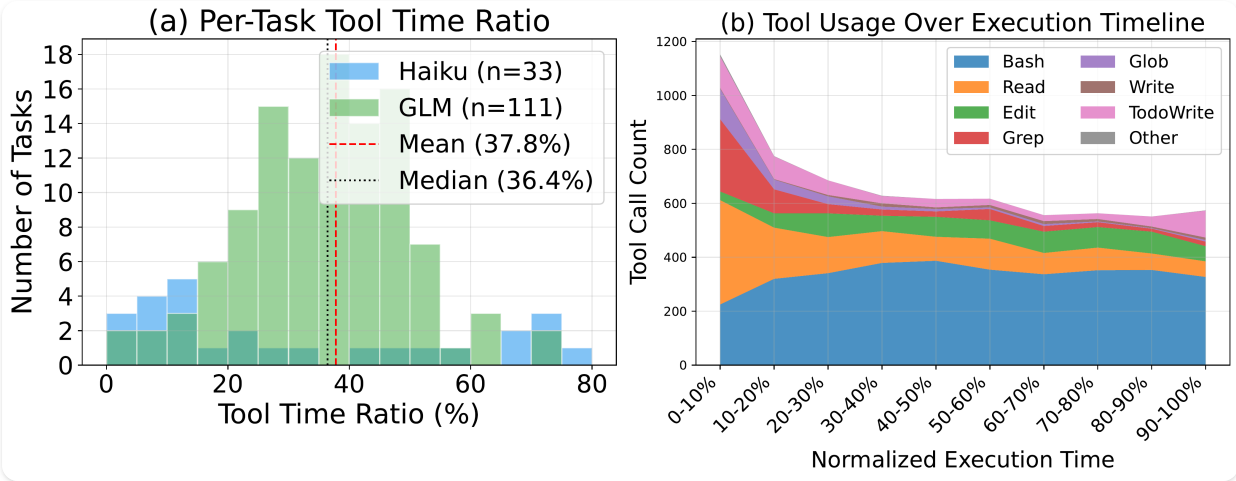
Q: Which tool types consume the most execution time?



- **Bash dominates tool execution, spanning three orders of magnitude in duration** (98.1% of tool time in GLM)
- Top categories: test execution **43.7%**, Python snippet 26.9%, package install 10.1%

Tool Execution: Temporal Pattern

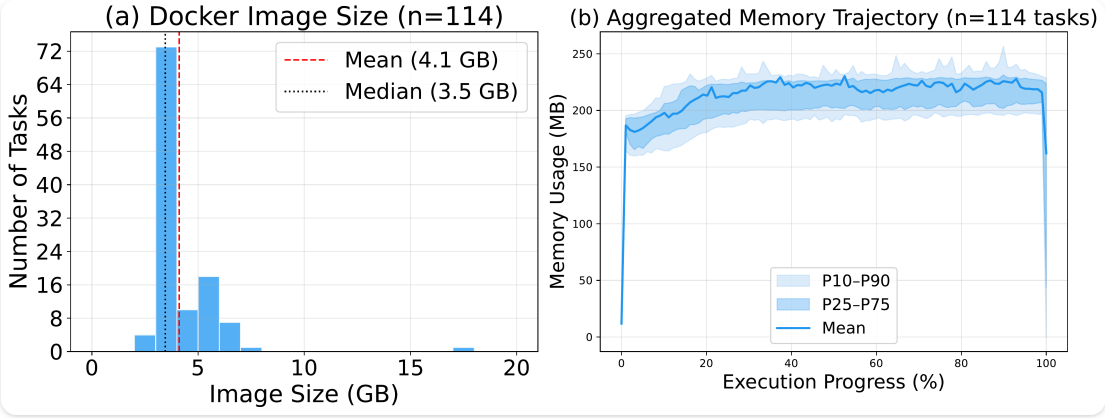
Q: When do different tools execute during a task?



- Tool time ratio: mean **37.8%**, median 36.4% of total task time
- Tool calls follow an "**understand, modify, verify**" pattern: Read in first 30%, Bash peaks at 40–80%

Memory & Resource Structure

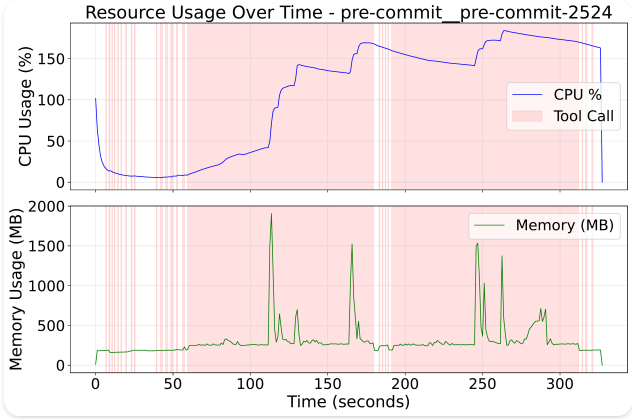
Q: What does resource consumption look like?



- **Memory, not CPU, is the primary bottleneck for multi-tenant concurrency density** (CPU avg <13%, memory peaks 2–4 GB)
- **Resource consumption exhibits a two-layer structure: a ~185 MB framework baseline plus tool-call bursts**

Resource Time Series

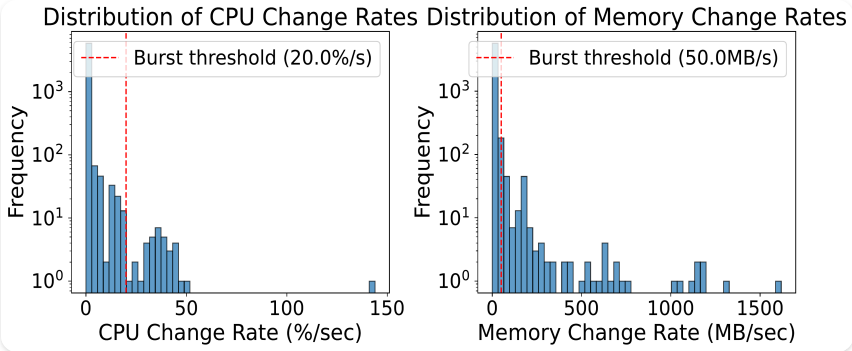
Q: What drives resource bursts?

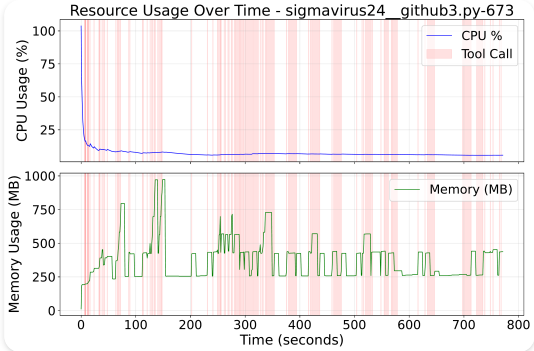


- **Within the burst layer, resource consumption is determined by what the tool *does* (e.g., pytest vs. git status), not which tool is invoked: Bash calls differ by 13.7× in peak memory**
- **Resource usage follows a burst-silence pattern, with 98.5% of memory bursts occurring during tool calls**

Burst Dynamics & Variability

Q: How fast and how variable are resource changes?





- **Resource bursts last 1–2 seconds with peak-to-average ratio up to 15.4×, several times beyond traditional cloud workloads**
- **85%–97% of tasks contain retry loops with progressive memory accumulation**
- **CPU-memory correlation varies by task (−0.84 to +0.50); co-directional change cannot be assumed**

Characterization Summary

Three key findings across 144 tasks:

1. **OS execution dominates**: 56–74% of latency is tool execution + initialization, not LLM reasoning
2. **Memory is the bottleneck**: CPU avg <13%, but memory peaks 2–4 GB with 15.4× peak/avg ratio
3. **Demands are unpredictable**: 20× across tasks, 1.8× across runs; CPU and memory are decoupled

Implication: resource management for AI agents must operate at **tool-call granularity**, react within **1–2 seconds**, and tolerate **non-determinism** — requirements no existing cloud tool was designed for.

Agent vs. Cloud Workloads

Q: How different from existing cloud workloads?

Dimension	Serverless/FaaS	Microservices	Batch/HPC	**AI Coding Agent**
Execution duration	100ms–2s	Long-running	Minutes–hours	**5–11 minutes**
Container image	~50 MB	100 MB–1 GB	1–10 GB	**2.9–17.3 GB (med. 3.5)**
Memory footprint	128–512 MB	Steady ~1 GB	Scales with data	**185 MB idle, peaks 2–4 GB**
Memory peak/avg	~1.5×	2–3×	~1×	**15.4×**
CPU utilization	Brief spike	10–40%	80–100%	**<13% avg, peaks >175%**
Determinism	Deterministic	Mostly deterministic	Deterministic	**1.8× variance same task**
Termination cost	Just retry	Can migrate	Lose progress	**Lose all LLM context**

Three Mismatches

Q: Why do existing resource management tools fail?

	Static Limits	Reactive Control	Predictive Scaling
Tools	mem.max/high, cpu.max; K8s QoS	PSI; oomd; TMO	VPA; Autopilot
Assumes	Known peak; stable demand	Gradual pressure; kill OK	Repeatable; history valid
Agent	15.4× peak/avg; tool-semantic	1–2s burst; unpredictable	1.8× variance; kill = lose context
Mismatch	**Granularity**	**Responsiveness**	**Adaptability**

- **Granularity**: `memory.max` at peak wastes >90%; at average → OOM
- **Responsiveness**: 3 GB/s bursts too fast for user-space tools (tens of ms reaction)
- **Adaptability**: Kill = triple penalty (cold start + lost context + re-execution)

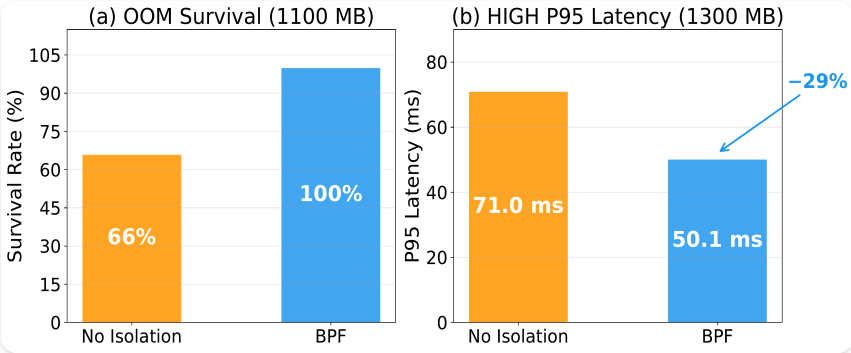
System Design

1. Fine-Grained Resource Domains → *Granularity*: transparent **bash wrapper** intercepts each `bash -c`, creates **ephemeral child cgroup** per tool call, collects metrics, removes on exit. No framework modification needed.

2. In-Kernel eBPF Enforcement → *Responsiveness*: `sched_ext` (CPU) + `memcg_bpf_ops` (memory) at kernel enforcement points. Graduated: **throttle** (`memory.high`) → **freeze** (`cgroup.freeze`) → **never kill**. Microsecond reaction.

3. Intent-Driven Resource Coordination → *Adaptability*: **upward** — agent declares need via `AGENT_RESOURCE_HINT` env var; **downward** — wrapper sends NL feedback via stderr when constrained → agent retries with lighter approach.

Evaluation



3 concurrent agents, replayed traces at 50×

OOM-free: all agents survive under memory pressure — no kills needed

Priority-aware: high-priority agents get lower latency under contention

Practical: wrapper overhead negligible, throttle precision validated

Takeaway: graduated enforcement (throttle → freeze → never kill) works

Limitations & Future Work

Limitations

- Requires patched kernel (`memcg_bpf_ops` under review)
- Evaluated with 3 agents; production runs 50–100+
- Bash-only interception

Future Work

- Larger-scale multi-tenant evaluation
- Upstream `memcg_bpf_ops` into mainline Linux
- Cross-agent scheduling (OpenClaw, Claude Code, Codex)
- Predictive hints from tool-call history

Thank You

Questions?

References

AgentCgroup Paper (arXiv:2602.09345) · AgentCgroup GitHub